micrograd-from-scratch

Scalar autodiff engine plus a small MLP on top of it, written from an empty file while going through video 1 of Karpathy's Neural Networks: Zero to Hero.

No frameworks. Every gradient in here is computed by code in this repo.

Layout

Everything is in `mlpProject.ipynb`, in this order:

- `Value`(the autodiff engine)
- `draw_dot`(renders the computation graph)
- `Neuron` / `Layer` / `MLP`(classes)
- the XOR training loop
- four experiment cells(commented out)

Each experiment cell is the training loop with one thing that's been changed. Uncomment one, run it, and you should get roughly what's written up below ^ ^

Value

Wraps a number and records how and where it came from.

a = Value(2.0)

b = Value(-3.0)

c = a * b

c.backward()

a.grad # -3.0

b.grad # 2.0

`__add__`, `__sub__` and `__mul__` are overloaded, so writing normal arithmetic builds the graph as a side effect. Every op attaches a closure to its output node holding the derivative rule for that op, with the parent nodes captured inside it.

`backward()` topologically sorts the graph depth-first, sets the output gradient to 1.0, then walks the ordering backwards calling the stored closures.

Gradients accumulate with `+=`.If a node feeds three consumers, all three send something back and the node needs the sum of them. Using `=` gets you whichever one happened to run last, which is a bad bug.

Operators: `+`, `-`, `*`, along with their reflected versions so `1 + a` works and not just `a + 1`, plus `pow()` and `tanh()`.

`pow()` and `tanh()` are methods rather than overloads, so `a.pow()` and `a.tanh()`. `pow()` has the exponent hardcoded to 2. The loss function is the only caller and it only ever squares, so I left it alone.

Network

n = MLP([2, 4, 1]) # 2 in -> 4 hidden -> 1 out

out = n([0.5, -1.0])

`Neuron` holds one weight per input plus a bias, all `Value` objects, and returns `tanh(w·x + b)`. `Layer` is a list of neurons fed the same input. `MLP` is a list of layers, each consisting of a list of neurons whose output it fed to the next.

`MLP([2, 4, 1])`has a total of 17 training parameters, and a single `backward()` call on the loss reaches all of them.

XOR

I've used XOR because a single linear layer can't do it, and I can show that in four lines:

w1*0 + w2*0 = -1 fixes the bias

w1*0 + w2*1 = 1 fixes w2

w1*1 + w2*0 = 1 fixes w1

w1*1 + w2*1 = -1 directly contradicts the two above

No assignment of weights satisfies all four and stacking more linear layers doesn't help much, since `3*(2*x)` is just `6*x`. So if this converges, the hidden layer and the tanh are doing something a linear model clearly cant.

Per step: forward pass over all four examples with the weights created only once across them, sum the four squared errors, zero the grads, `backward()`, update.

step 0: loss = 4.060138

step 20: loss = 2.000427

step 40: loss = 0.763530

step 60: loss = 0.340645

step 80: loss = 0.192634

step 100: loss = 0.127094

step 120: loss = 0.092244

step 140: loss = 0.071259

step 160: loss = 0.057479

step 180: loss = 0.047845

targets: [-1, 1, 1, -1]

predictions: [-0.904118, 0.922252, 0.889240, -0.884770]

I ran it 10 times from different random initializations. They converged every time, final loss somewhere between 0.03 and 0.07.

Experiments

1. Taking the tanh() out

`Neuron.__call__` returns the raw weighted sum.

step   0: loss = 4.372179

step 100: loss = 4.000000

step 200: loss = 4.000000

Sits at exactly 4.0 and never moves. I assumed it was stuck, but 4.0 is the actual least-squares optimum for a linear model on XOR. Best linear fit is all-zero weights, which predicts 0 everywhere, and 1+1+1+1 = 4. So it converged fine. It converged to the best answer available to a model that can't represent the problem, which was…a problem.

2. Learning rate of 10

Both update steps changed from `-0.05` to `-10`. This was the one I expected to blow up instantly

step 0: loss = 4.277563

step 20: loss = 8.000000

step 40: loss = 8.000000

predictions: [1.0, 1.0, 1.0, 1.0]

Apparently It doesn't blow up. No NaNs, nothing runs off to infinity. It literally stops on step one and then does nothing for the remaining 199 steps.

One update at this rate takes the largest weight from around 1 to somewhere past 12, and `tanh(12)` is 1.0 as far as float precision is concerned. The derivative is `1 - tanh(x)**2`, so once a unit is saturated its gradient is about 1e-10 and the weight can't move again. Everything downstream is dead with it.

The 8.0 is the network outputting 1.0 to everything. Two of the four targets are +1, so those come out right by accident; the other two are -1 and contribute 4 each.

3. Never zeroing the gradients

Deleted the zeroing pass.

```
step   0: loss = 5.578789
step  60: loss = 0.036448
step 120: loss = 0.000139
```

Converges *faster* than the correct version, 1e-4 by step 120 against roughly 0.09. The early gradients all point the same way, so the running sum keeps growing and the effective learning rate climbs on its own. It's momentum with no decay term and no ceiling.

Still a bug. Whether uncapped momentum helps or wrecks a run is decided by the initialization and you get no say in it. Hence `zero_grad()`.

4. Two hidden neurons instead of four

10 runs each, different random inits:

width 4: 10/10 converged (0.03 - 0.07)

width 2: 6/10 converged (0.05 - 0.196)

4/10 plateaued (2.12 - 2.68)

Two is the theoretical minimum for XOR: one neuron detecting AND, one detecting OR, output layer subtracting them. The solution is in there. But there's essentially one weight configuration that implements it, so gradient descent either walks into it or wanders into a flat region and stops.

Width 4 doesn't make XOR easier. It gives the optimizer more paths to a working solution, so the run stops depending on where the init lands.

Worth separating this one from the other three, since nothing here is actually broken. A width-2 network can solve XOR and usually does. What changes is how often.

Notes

Squared error rather than absolute error. The loss value itself is never used for anything, only its derivative, and squaring makes that derivative proportional to the error with the sign handled for you. Absolute error gives ±1 regardless of magnitude, so a badly wrong prediction would get corrected exactly as hard as a nearly-right one.

Subtraction has its own node. `__sub__` builds a `-` node whose backward rule sends +1 to the left operand and -1 to the right. You don't strictly need it: `a - b` could be `a + (b * -1)` and the gradient falls out of the existing add and multiply rules for free. Anything you can write with `+` and `*` needs no new rule at all. I wrote it explicitly anyway.

`__rsub__` is wrong. It returns `self - other`, so `2 - a` evaluates as `a - 2`. Nothing in this repo notices, because every subtraction ends up inside `pow()` and `(a-b)**2 == (b-a)**2` with identical gradients either way. Fixing it properly means adding a `__neg__`, which I haven't.

Running it

Open the notebook and run top to bottom. Ive commented out the main experiment cells so the main XOR run goes through clean; uncomment one at a time.

Needs graphviz for the graph drawings. numpy and matplotlib get imported in the first cell because I was plotting things while building this, and I never took the imports back out.

Credit

I followed Karpathy's Neural Networks: Zero to Hero, video 1(The spelled-out intro to neural networks and backpropagation: building micrograd). Code was written from scratch and compared against his afterwards. The XOR training, the experiments and the write-ups are mine.